FACULTY OF ARTS

KU LEUVEN

Computational Linguistics

Lecture 4: Neural architectures for sentence processing

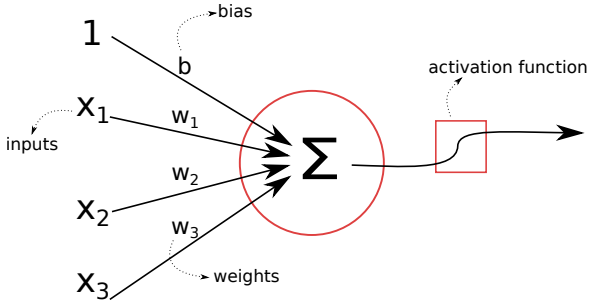
Tim Van de Cruys



Tuesday 20 October 2026

Reminder

- Neuron = logistic regression

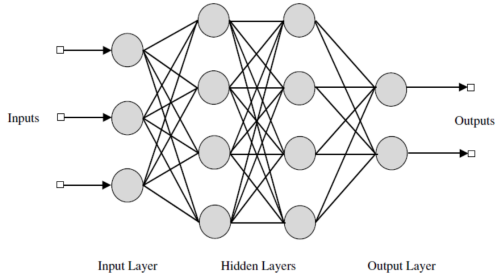


- We can feed an input vector to a bunch of logistic regression functions and get an output vector
- which can be fed to another layer of logistic regression functions

Feed-forward

Architecture

Multi-layer perceptron with 2 hidden layers



$$NN_{MLP2}(\mathbf{x}) = \mathbf{y} \quad (1)$$

$$\mathbf{h}^1 = g(\mathbf{x}\mathbf{W}^1 + \mathbf{b}^1) \quad (2)$$

$$\mathbf{h}^2 = g(\mathbf{h}^1\mathbf{W}^2 + \mathbf{b}^2) \quad (3)$$

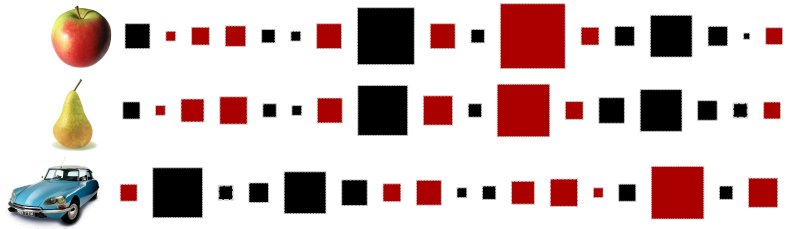
$$\mathbf{y} = \mathbf{h}^2\mathbf{W}^3 \quad (4)$$

$\mathbf{W}^1, \mathbf{W}^2, \mathbf{W}^3, \mathbf{b}^1, \mathbf{b}^2$ = parameters of the network (θ)

Use of multiple hidden layers: **deep learning**

Word embeddings

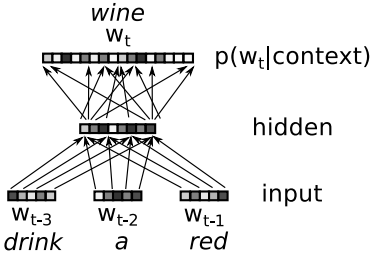
- Each word i is represented by a small, dense vector $\mathbf{v}_i \in \mathbb{R}^d$
- d is typically in the range 50–1000
- Matrix of size V (vocabulary size) $\times$ d (embedding size)
- words are 'embedded' in a real-valued, low-dimensional space
- Similar words have similar embeddings



Feed-forward network

Language modeling

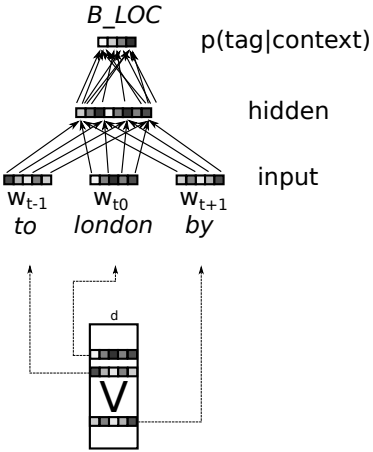
- Predict the next word in a sequence, based on the previous word
- One non-linear hidden layer, one softmax layer for classification
- Choose parameters that optimize probability of correct word



Feed-forward network

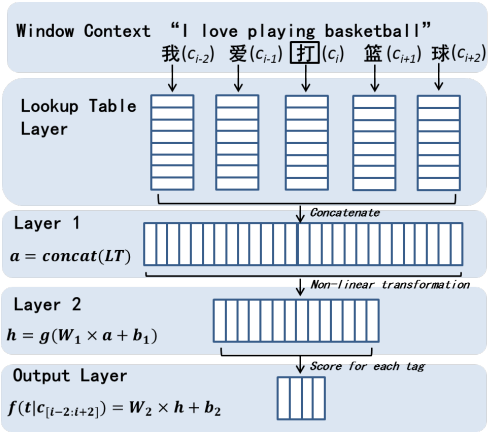
Named entity recognition

- Predict named entity tag for current word, based on current word and previous + next word
- One non-linear hidden layer, one softmax layer for classification
- Choose parameters that optimize probability of correct word



Feed-forward network

Word segmentation



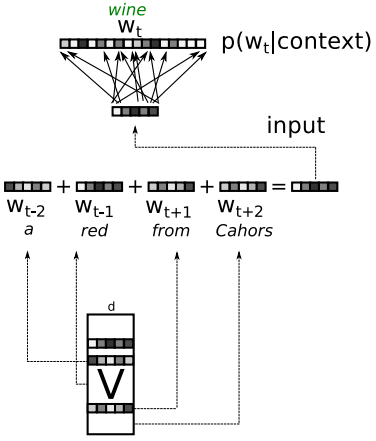
Pei et al. (2014)

Encoding sentences

- Feed-forward network assumes fixed dimensional input
- How to represent variable number of features, e.g. words in a sentence, document?
  - Continuous Bag of Words (CBOW): sum embedding vectors of corresponding features
    - no ordering info (“not good quite bad” = “not bad quite good”)
  - Recurrent architecture
    - Recurrent connection to encode sentence history
  - Transformer architecture
    - Self-attention

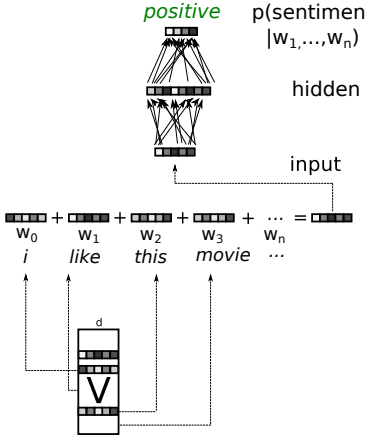
CBOW: induction of embeddings

- Current word w_t is predicted from context words
- Prediction is made from the sum of context embeddings
- Pairwise sum of values of corresponding embedding features



CBOW: sentiment classification

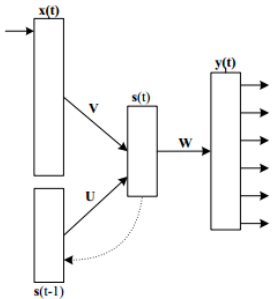
- Sentiment prediction based on sentence/document words
- Prediction is made from the sum of context embeddings
- Pairwise sum of values of corresponding embedding features
- Works for variable number of words



Recurrent neural network

Introduction

- CBOW: no ordering, no structure
- RNN: represent arbitrarily sized structured input as fixed-size vectors, paying attention to structured properties



Recurrent neural network

Model

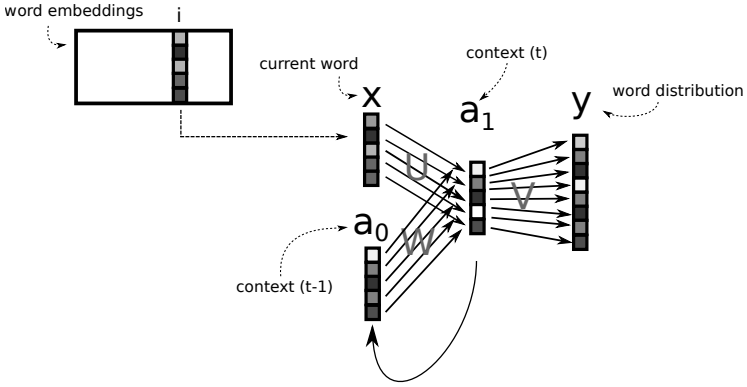
- $\mathbf{x}_1$: input layer (current word)
- $\mathbf{a}_1$: hidden layer of current timestep
- $\mathbf{a}_0$: hidden layer of previous timestep
- $\mathbf{U}$, $\mathbf{W}$ and $\mathbf{V}$: weights matrices
- $f(\cdot)$: element-wise activation function (sigmoid)
- $g(\cdot)$: softmax function to ensure probability distribution

$$\mathbf{a}_1 = f(\mathbf{U}\mathbf{x}_1 + \mathbf{W}\mathbf{a}_0) \tag{5}$$

$$\mathbf{y}_1 = g(\mathbf{V}\mathbf{a}_1) \tag{6}$$

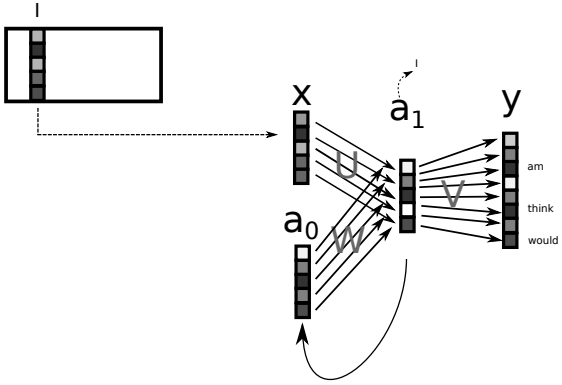
Recurrent neural network

Language model



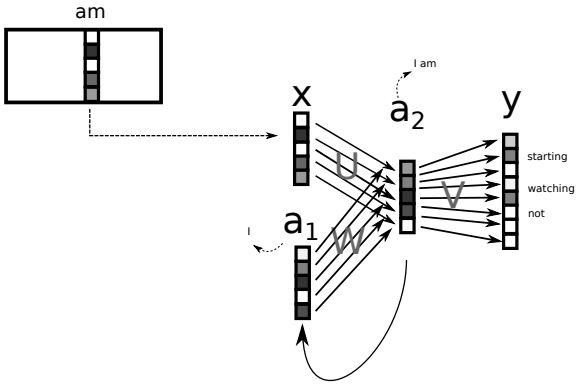
Recurrent neural network

Language model



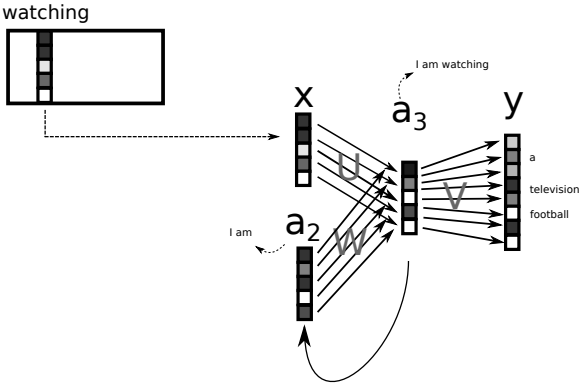
Recurrent neural network

Language model



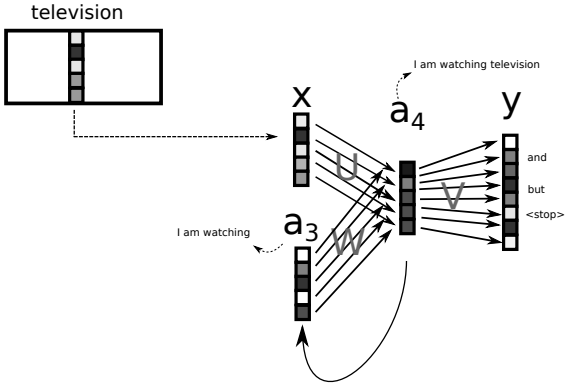
Recurrent neural network

Language model



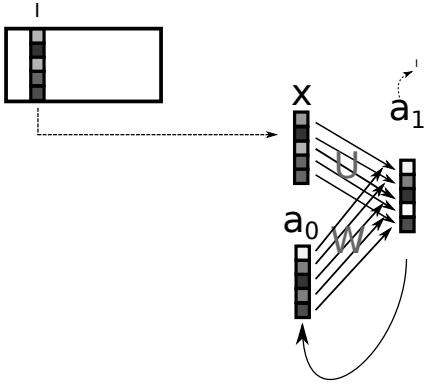
Recurrent neural network

Language model



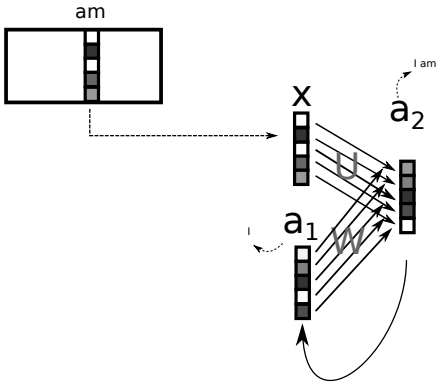
Recurrent neural network

Language model = sentence encoder



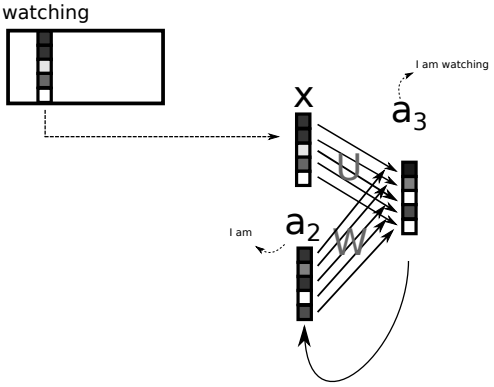
Recurrent neural network

Language model = sentence encoder



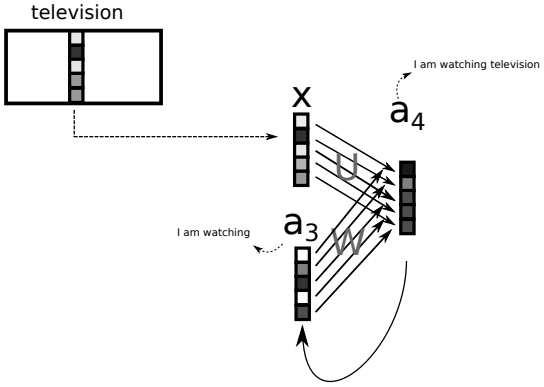
Recurrent neural network

Language model = sentence encoder



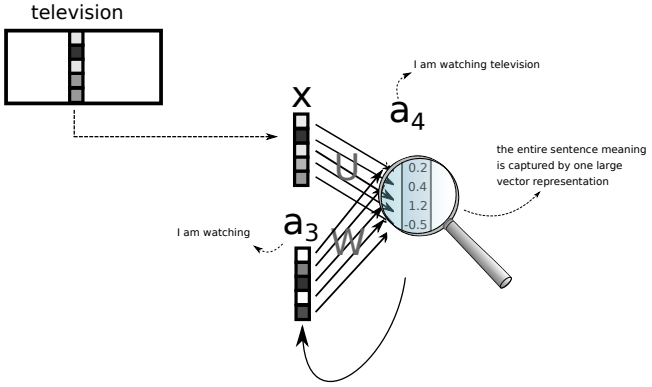
Recurrent neural network

Language model = sentence encoder



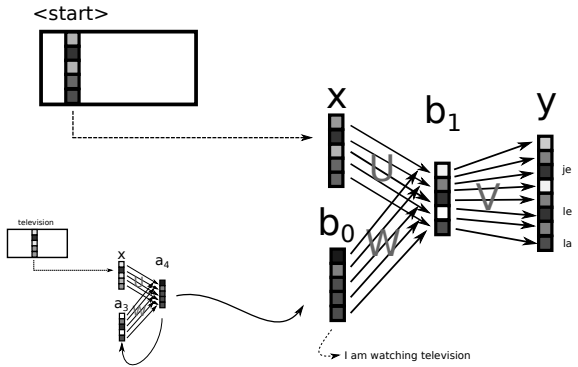
Recurrent neural network

Language model = sentence encoder



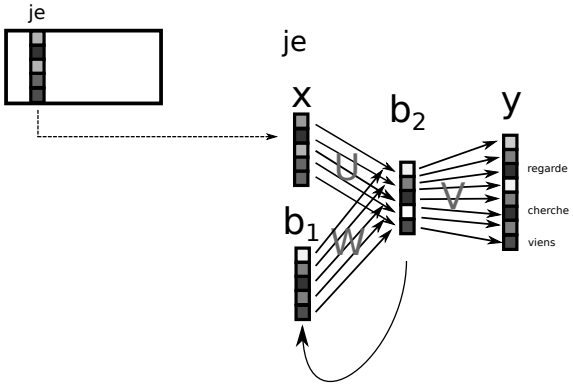
Recurrent neural network

Encoder-decoder



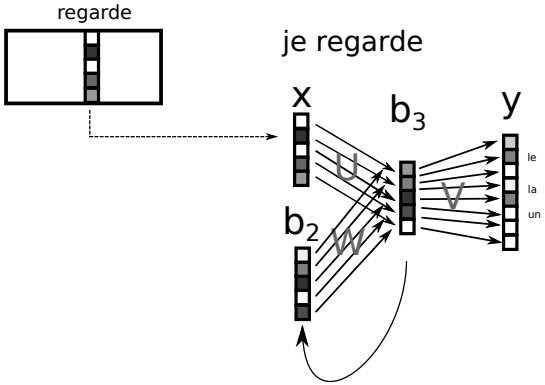
Recurrent neural network

Encoder-decoder



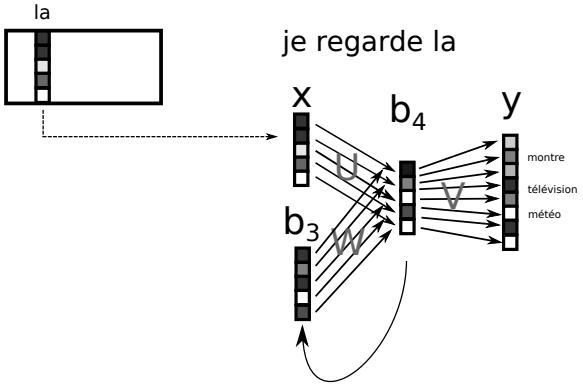
Recurrent neural network

Encoder-decoder



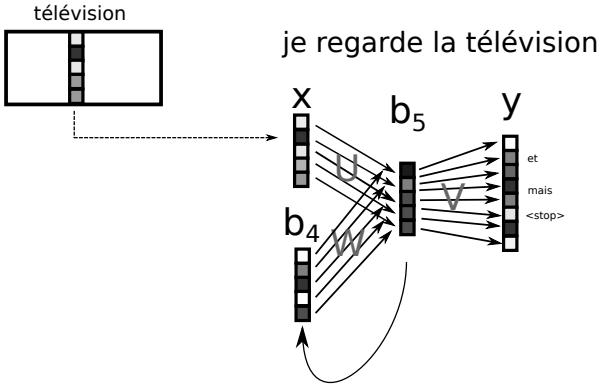
Recurrent neural network

Encoder-decoder



Recurrent neural network

Encoder-decoder



Recurrent neural network

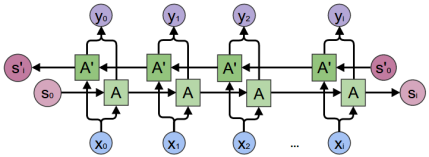
Training

- Consider recurrent neural network as very deep neural network with shared parameters across computation
- Backpropagation through time
- What kind of supervision?
  - Acceptor: based on final state
  - Transducer: an output for each input (e.g. language modeling)
  - Encoder-decoder: one RNN to encode sequence into vector representation, another RNN to decode into sequence (e.g. machine translation)

Recurrent neural network

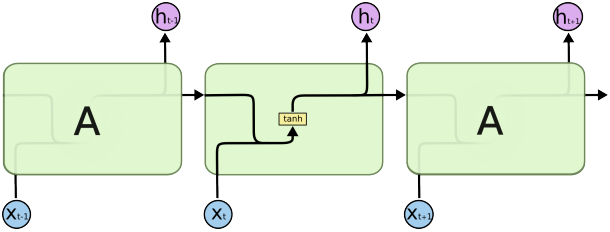
Bi-directional RNN

- Input sequence both forward and backward to different RNNs
- Representation is concatenation of forward and backward state (A & A')
- Represent both history and future



Concrete RNN architectures

Simple RNN

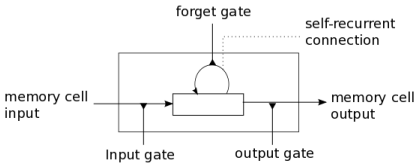


source images: `colah.github.io`

Concrete RNN architectures

LSTM

- Long short term memory networks
- In practice, simple RNNs only able to remember narrow context (vanishing gradient)
- LSTM: complex architecture able to capture long-term dependencies



Recurrence: difficulties

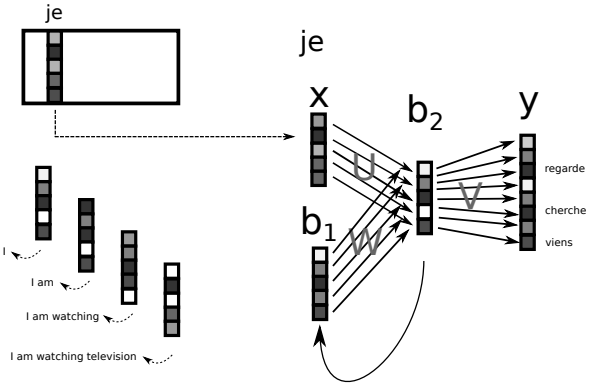
- RNNs process a sentence word by word
- information from the beginning of the sequence needs to survive many processing steps
- long-distance dependencies are hard to capture in practice
- no parallel computation: each step waits for the previous one

Bottleneck

- In standard encoder-decoder approach, the entire meaning of a sentence is encoded into a single vector
- Vector is always the same length, no matter how long the sentence
  - Can the meaning of a sentence be represented by a single vector?
  - Does it make sense to model a two-word sentence and a thirty-word sentence with the same vector size?
- Solution: for each prediction in the decoder phase, pay **attention** to the entire range of intermediate encoder states

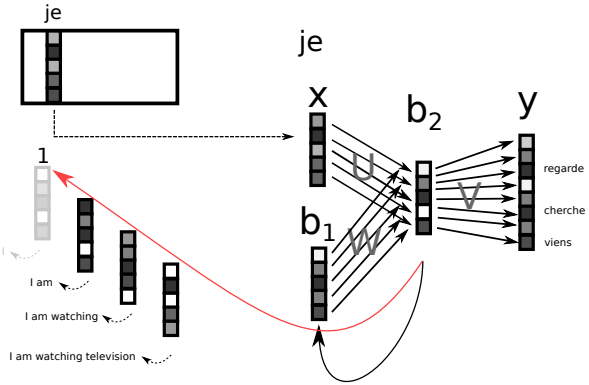
Recurrent neural network

Attention



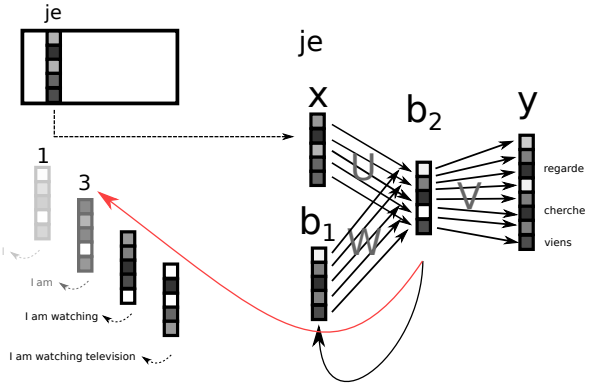
Recurrent neural network

Attention



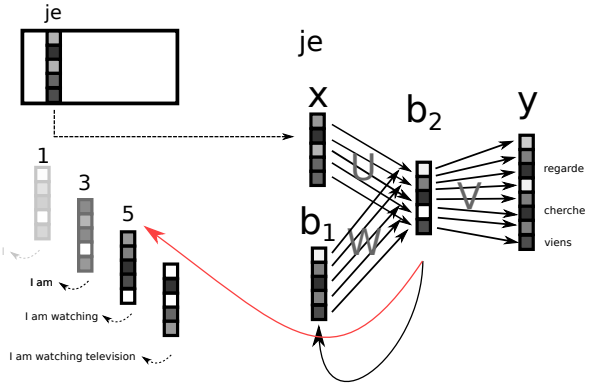
Recurrent neural network

Attention



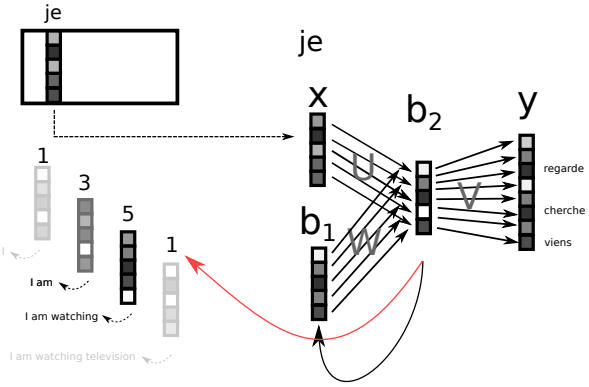
Recurrent neural network

Attention



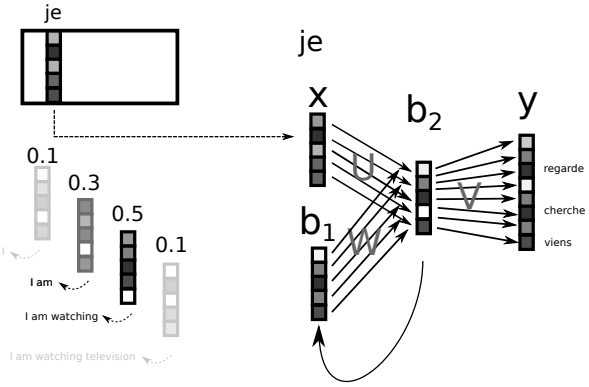
Recurrent neural network

Attention



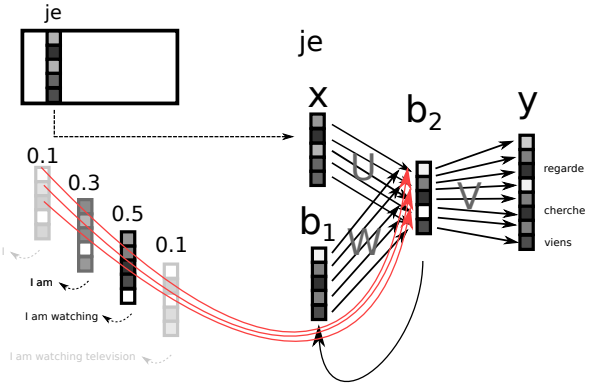
Recurrent neural network

Attention

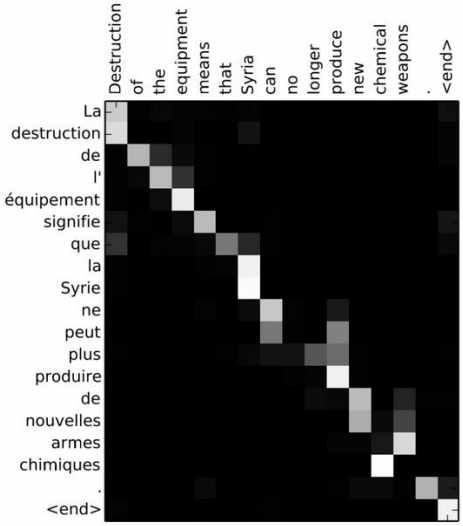


Recurrent neural network

Attention



Attention in machine translation



Attention

The key idea

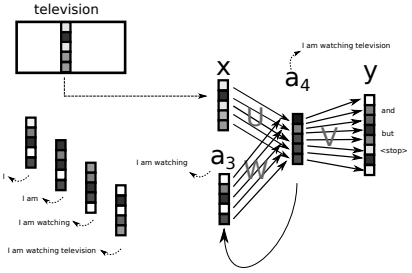
- Attention computes a **weighted average** of a set of representations (relevance of each representation for the current prediction)
- We can use attention within an RNN encoder-decoder framework, as a fix for the bottleneck
- But attention is a general mechanism (and can replace recurrence altogether)

Contextualized word embeddings

- Standard word embeddings are static: a single embedding for each word
- But words are polysemous
- And each word occurrence has a slightly different meaning in each different context
- **contextualized word embeddings**
  - ELMo: Embeddings from Language Models
  - BERT: Bidirectional Encoder Representations from Transformers

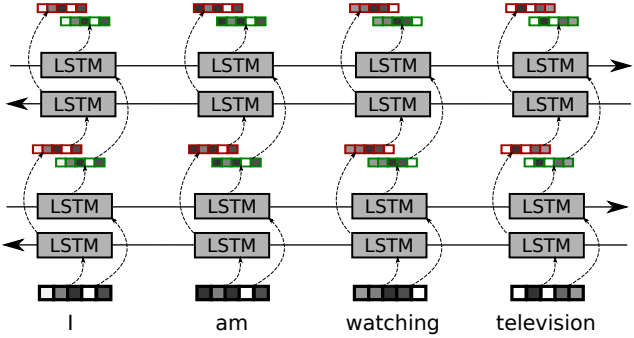
ELMo

- Language models are good at the induction of embeddings
- Let's use the intermediate hidden states of an RNN as contextualized word embeddings
- Use contextualized embeddings instead of static embedding in task-specific neural network architecture
- Quickly superseded by transformer models, but the idea of contextualized embeddings remains central

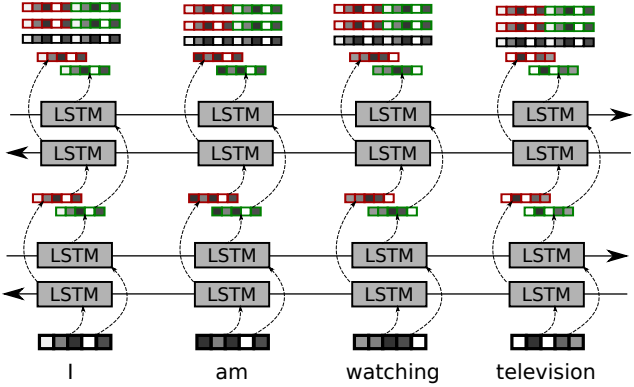


Peters et al. (2018)

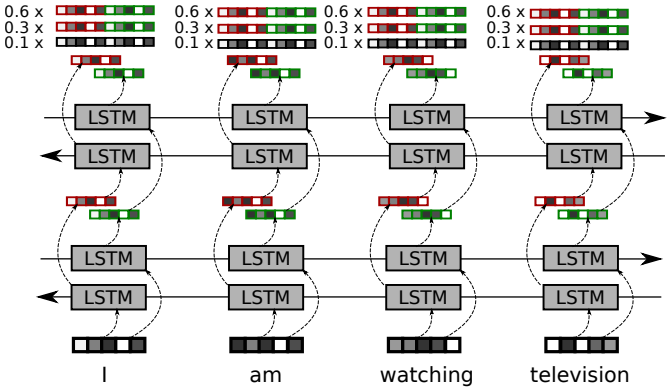
ELMo


LSTM
LSTM
LSTM
LSTM
LSTM
LSTM
LSTM
LSTM
LSTM
LSTM
LSTM
LSTM
I
am
watching
television

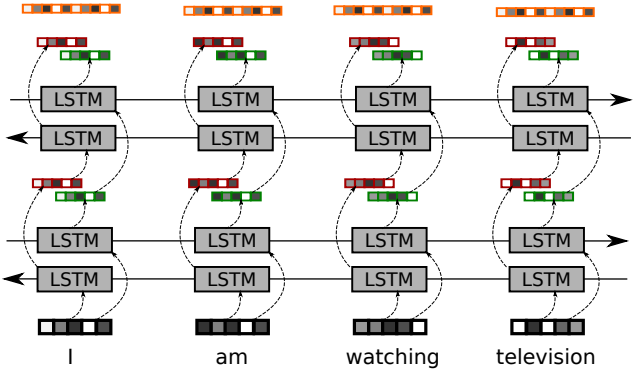
ELMo


LSTM
LSTM
LSTM
LSTM
LSTM
LSTM
LSTM
LSTM
LSTM
LSTM
LSTM
LSTM
I
am
watching
television

ELMo



ELMo


LSTM
LSTM
LSTM
LSTM
LSTM
LSTM
LSTM
LSTM
LSTM
LSTM
LSTM
LSTM
I
am
watching
television

Attention is all you need

- ELMo still relies on recurrence: slow, sequential, limited context
- Radical idea: drop the recurrence, keep only the attention
- The resulting architecture is called the **transformer**
  - originally developed for machine translation
  - now the basic building block of today's LLM architectures

Vaswani et al. (2017). Attention is all you need

Transformer

- Basic building block of LLMs
- Like a fancy feedforward network, based on attention mechanism
- Transformer computes **contextualized** embeddings for words, based on surrounding words

`The astronomer watched the` **star**

`The audience watched the` **star**

- Used for **left to right** language modeling
- But first we'll look at **bi-directional language modeling**

Transformer

- Basic building block of LLMs
- Like a fancy feedforward network, based on attention mechanism
- Transformer computes **contextualized** embeddings for words, based on surrounding words

`The astronomer watched the` **`star`** `through the telescope.`

`The audience watched the` **`star`**

- Used for **left to right** language modeling
- But first we'll look at **bi-directional language modeling**

Transformer

- Basic building block of LLMs
- Like a fancy feedforward network, based on attention mechanism
- Transformer computes **contextualized** embeddings for words, based on surrounding words

`The astronomer watched the` **`star`** `through the telescope.`

`The audience watched the` **`star`** `in awe.`

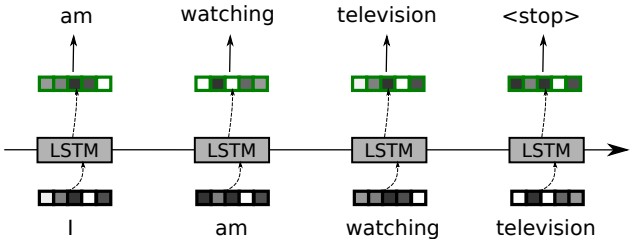
- Used for **left to right** language modeling
- But first we'll look at **bi-directional language modeling**

BERT

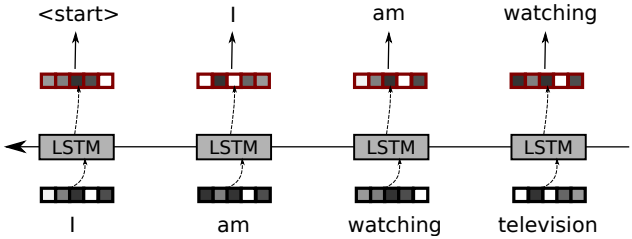
- NLP model that yielded state of the art results
- Representations based on bi-directional context
- Transformer architecture
- General training on language modeling task, finetuning on specific NLP task

Devlin et al. (2019)

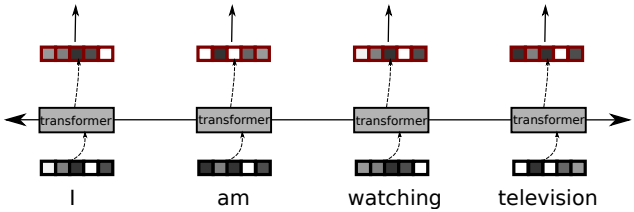
Bi-directional encoding



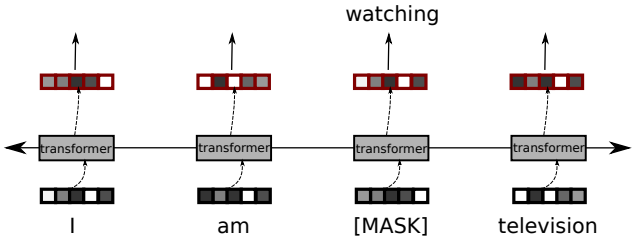
Bi-directional encoding



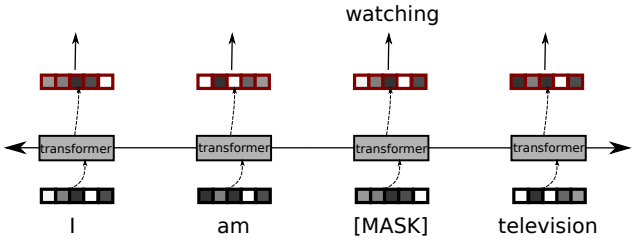
Bi-directional encoding



Bi-directional encoding



Bi-directional encoding

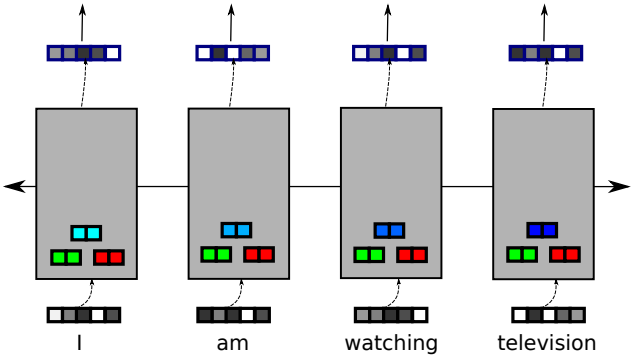


- Mask 15% of input words, and predict the masked words

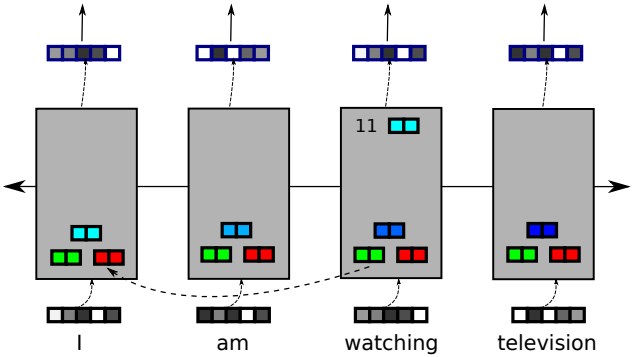
Transformer

- Computation of contextualized representations based solely on concept of **self-attention**
- No more sequential processing, allows for efficient parallel computations
- Representation of each word is contextualized through application of multiple layers of transformer blocks

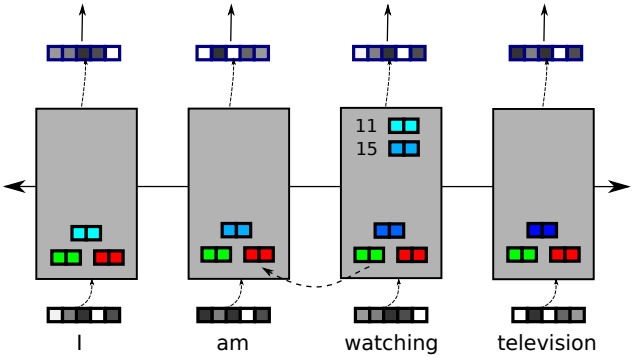
Transformer: self-attention



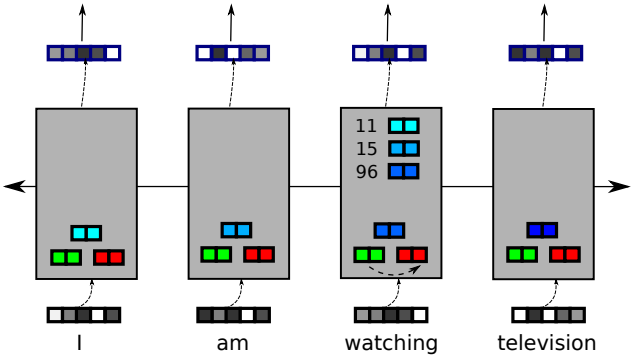
Transformer: self-attention



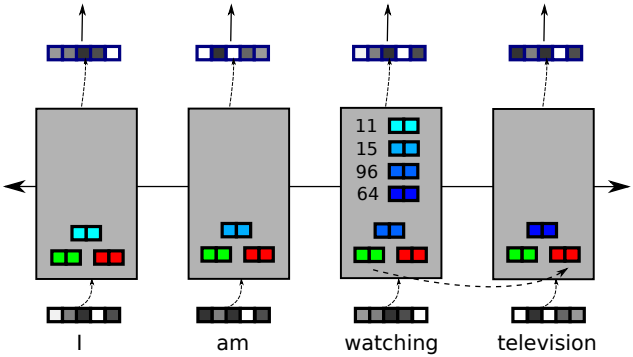
Transformer: self-attention



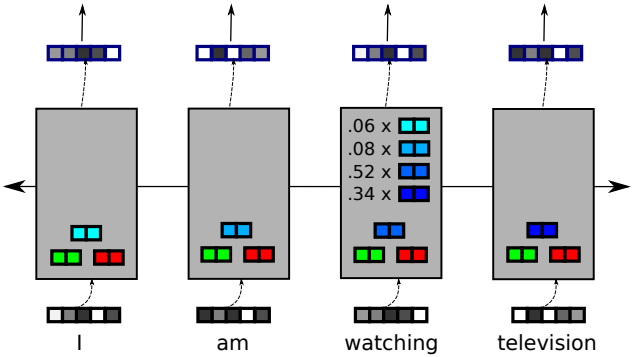
Transformer: self-attention



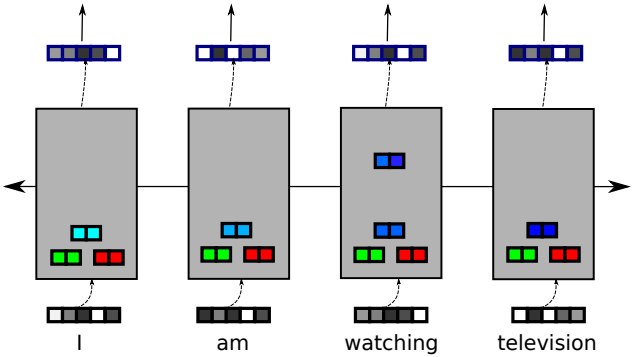
Transformer: self-attention



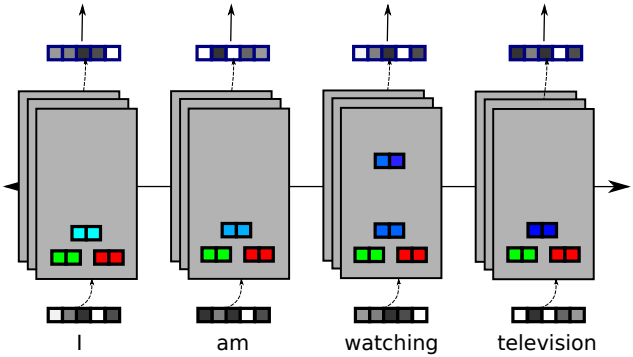
Transformer: self-attention



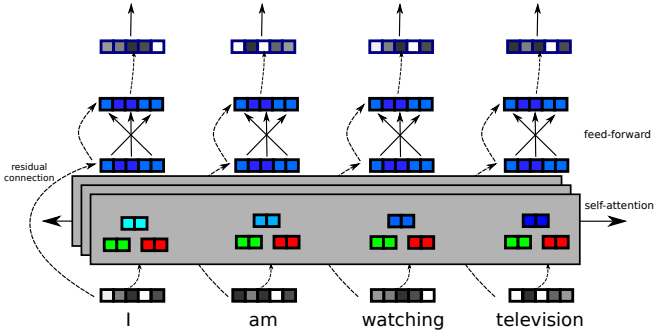
Transformer: self-attention



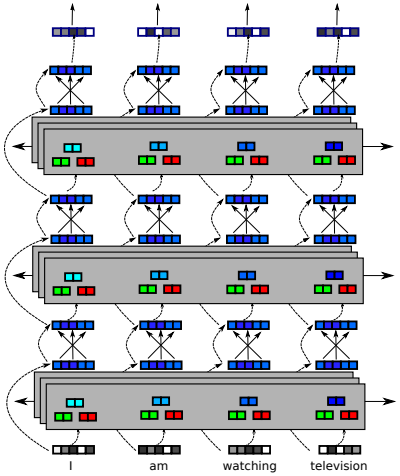
Transformer: self-attention



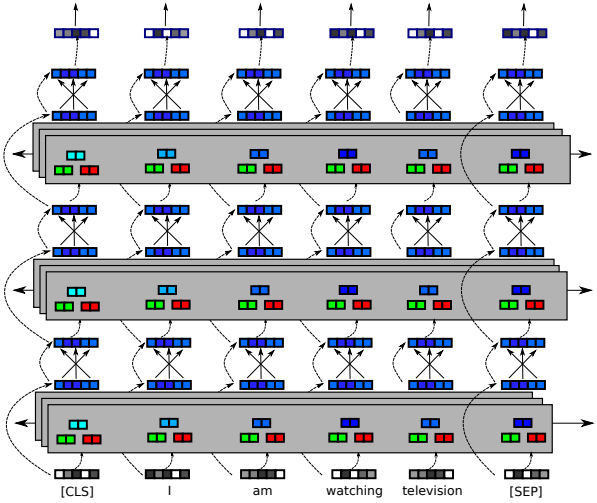
BERT


feed-forward
residual
connection
self-attention
I
am
watching
television

BERT



BERT



Transformer

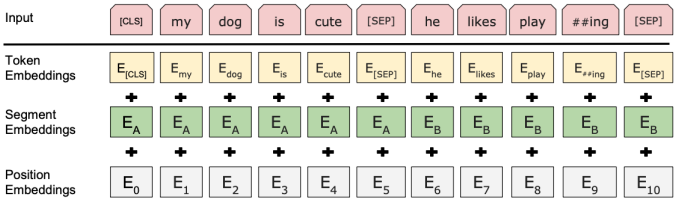
Subword units

- As a vocabulary, BERT uses subword units, automatically induced using subword segmentation algorithm called **WordPiece**
- Two reasons to use subwords:
  - Smaller vocabulary = faster computations
  - No out of vocabulary words: combination of subwords
- Note: subword units *can* be entire words
- Algorithm:
  - Initialize vocabulary with all characters of training corpus
  - Iteratively merge characters/substrings that are the most likely according to the data
  - Continue until predefined vocabulary limit is reached
- e.g., `I am watch ##ing television`

Transformer

Positional encodings

- Due to self-attention, the order of the words within the sentence is lost
- This is solved by including **positional encodings**
- Additional vector that represents position in the sentence
- Added to the initial embeddings



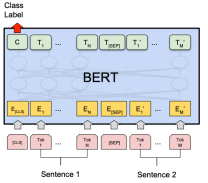
BERT

Pre-training and finetuning

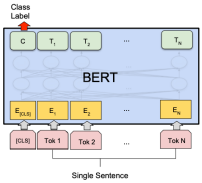
- BERT is pre-trained on the masked language modeling task using a large corpus
- Once the model is pretrained for language modeling, it can be finetuned for a special natural language processing task
- Special, reserved token <CLS> is added to the start of each sentence
- The resulting embedding is considered as a representation of the entire sentence
- The representation can be used to perform a final classification task (by adding a softmax classification layer on top)
- The representation can be used as is, but the most common practice is to **finetune all the parameters of the model** to the task at hand

BERT

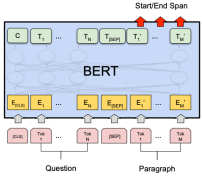
For NLP tasks



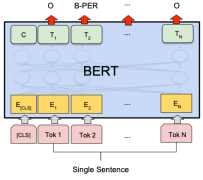
(a) Sentence Pair Classification Tasks: MNLI, QQP, QNLI, STS-B, MRPC, RTE, SWAG



(b) Single Sentence Classification Tasks: SST-2, CoLA



(c) Question Answering Tasks: SQuAD v1.1



(d) Single Sentence Tagging Tasks: CoNLL-2003 NER

BERT

Results

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	**Average** -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.9	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	88.1	91.3	45.4	80.0	82.3	56.0	75.2
$\text{BERT}_{\text{BASE}}$	84.6/83.4	71.2	90.1	93.5	52.1	85.8	88.9	66.4	79.6
$\text{BERT}_{\text{LARGE}}$	**86.7/85.9**	**72.1**	**91.1**	**94.9**	**60.5**	**86.5**	**89.3**	**70.1**	**81.9**

Model	**data**	**bsz**	**steps**	**SQuAD** (v1.1/2.0)	**MNLI-m**	**SST-2**
RoBERTa						
with BOOKS + WIKI	16GB	8K	100K	93.6/87.3	89.0	95.3
+ additional data (§3.2)	160GB	8K	100K	94.0/87.7	89.3	95.6
+ pretrain longer	160GB	8K	300K	94.4/88.7	90.0	96.1
+ pretrain even longer	160GB	8K	500K	**94.6/89.4**	**90.2**	**96.4**
$\text{BERT}_{\text{LARGE}}$						
with BOOKS + WIKI	13GB	256	1M	90.9/81.8	86.6	93.7
$\text{XLNet}_{\text{LARGE}}$						
with BOOKS + WIKI	13GB	256	1M	94.0/87.8	88.4	94.4
+ additional data	126GB	2K	500K	94.5/88.8	89.8	95.6

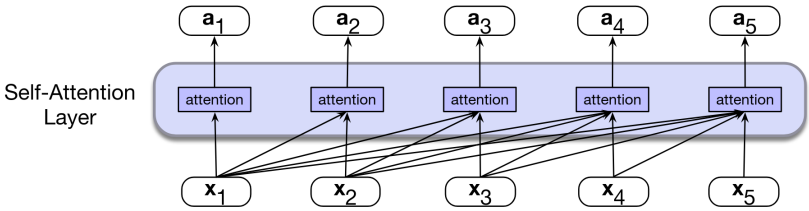
Wang et al. (2018): GLUE Multi-Task Benchmark

Transformer for Language Generation

- Transformer encoder works well for the induction of language representations, but the bi-directional encoding is less suitable for language generation
- How can a transformer be used for language generation?
- No attention to future positions, only attend to previous positions

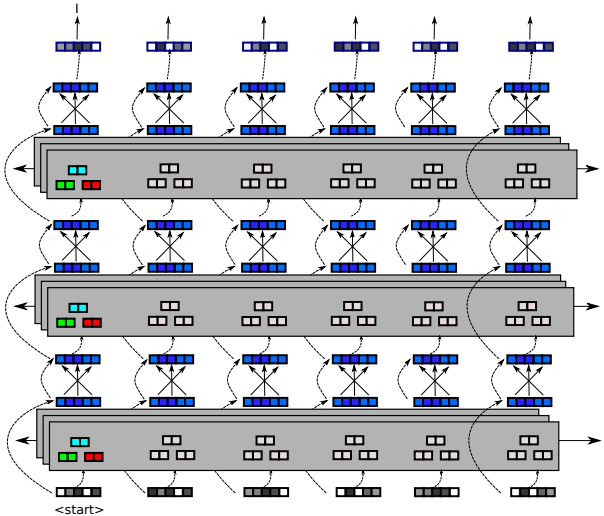
Causal attention

- Attention in causal language modeling (left to right)
- When processing $\mathbf{x}_i$, we attend to all inputs up to and including $\mathbf{x}_i$

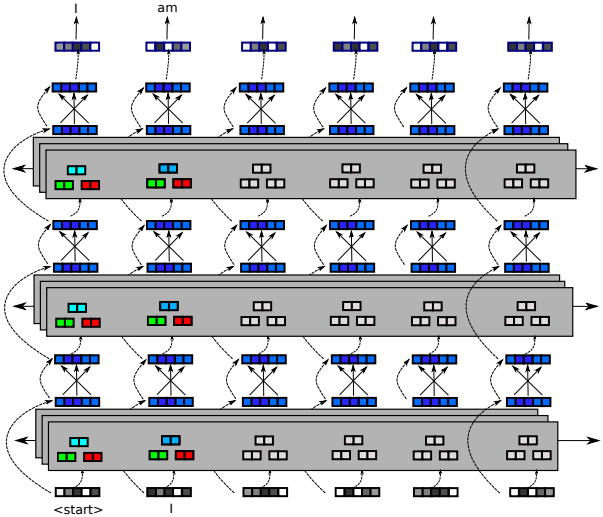


Jurafsky and Martin (2024)

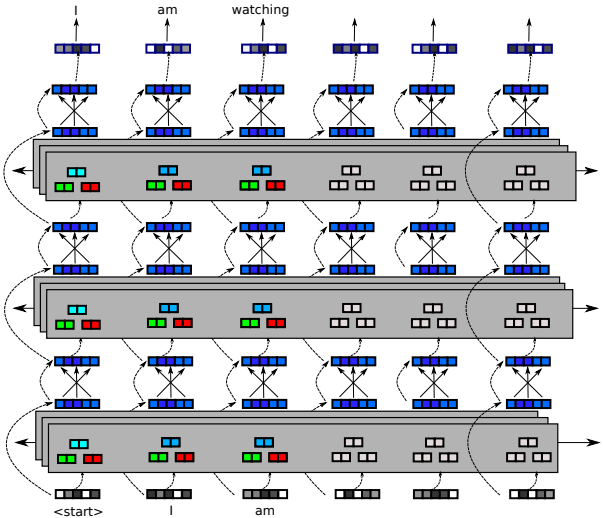
Transformer for Language Generation



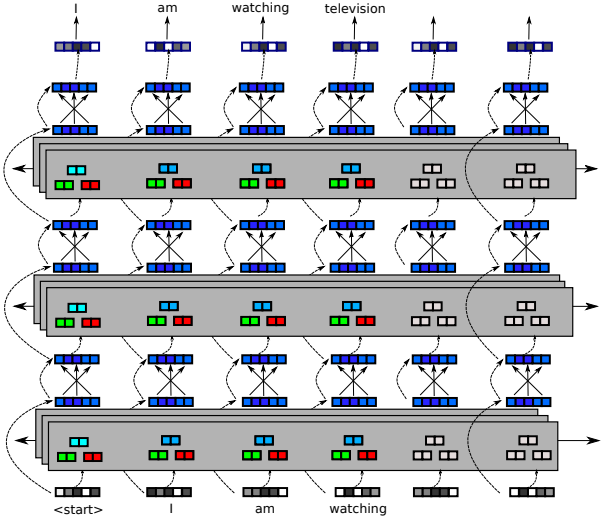
Transformer for Language Generation



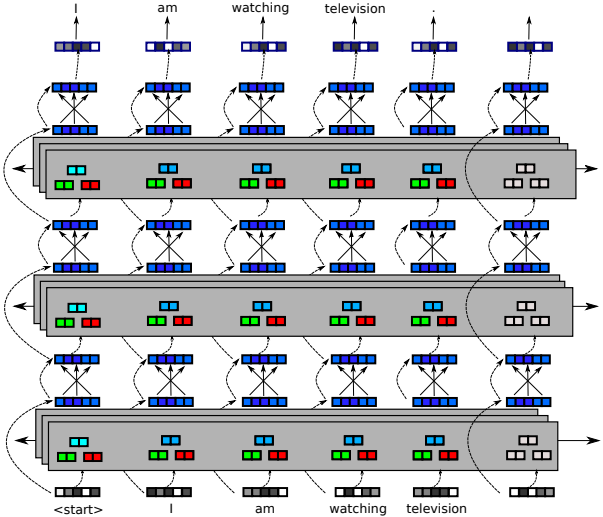
Transformer for Language Generation



Transformer for Language Generation



Transformer for Language Generation

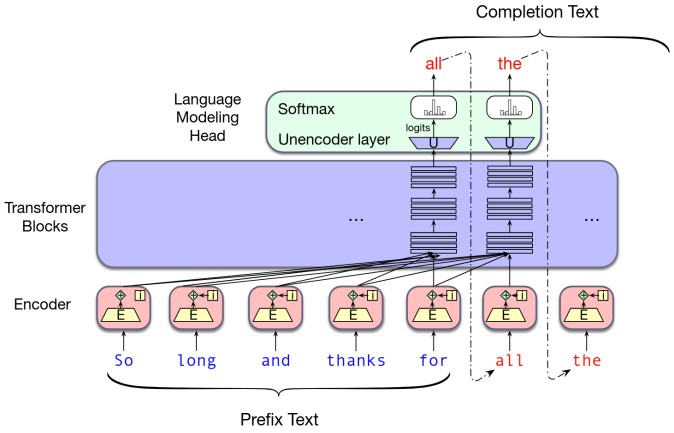


GPT

- Generative Pre-trained Transformer
- Pre-trained on next-word prediction over large corpora
- Different versions:
  - GPT: 117 million parameters
  - GPT–2: 1.5 billion parameters
  - GPT–3: 175 billion parameters
  - ...

Brown et al. (2020). Language models are few-shot learners

Conditional generation



Jurafsky and Martin (2024)

The power of scale

- LLMs: transformer architecture, applied on a large scale
- Lots of parameters, lots of data
- Just by predicting words, LLMs learn to solve many tasks
  - Sentiment analysis
  - Question answering
  - Summarization
  - ...

Examples

- p(w|*The sentiment of the sentence 'The new Bond movie is fantastic' is:*)
- p(w|*The capital of France is*)

Summarization

The only thing crazier than a guy in snowbound Massachusetts boxing up the powdery white stuff and offering it for sale online? People are actually buying it. For $89, self-styled entrepreneur Kyle Waring will ship you 6 pounds of Boston-area snow in an insulated Styrofoam box – enough for 10 to 15 snowballs, he says. But not if you live in New England or surrounding states.

"We will not ship snow to any states in the northeast!" says Waring's website, ShipSnowYo.com. "We're in the business of expunging snow!"

His website and social media accounts claim to have filled more than 133 orders for snow – more than 30 on Tuesday alone, his busiest day yet. With more than 45 total inches, Boston has set a record this winter for the snowiest month in its history.

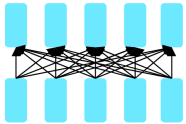
According to Boston.com, it all started a few weeks ago, when Waring and his wife were shoveling deep snow from their yard in Manchester-by-the-Sea, a coastal suburb north of Boston. He joked about shipping the stuff to friends and family in warmer states, and an idea was born.

Summary:

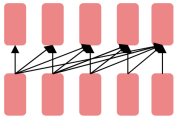
Kyle Waring will ship you 6 pounds of Boston-area snow in an insulated Styrofoam box – enough for 10 to 15 snowballs, he says. But not if you live in New England or surrounding states.

Three architectures for LLMs

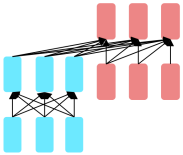
- Encoder
  - BERT: masked language modeling
  - Predict words based on left and right context
  - Usually fine-tuned with supervised data



- Decoder
  - GPT
  - Word prediction (and many tasks that can be framed as word prediction)



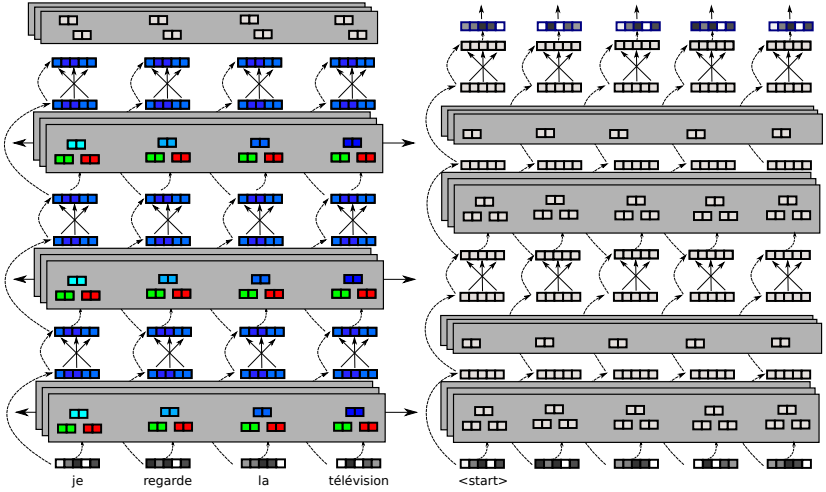
- Encoder-Decoder
  - T5
  - Map from one sequence to another
  - machine translation, speech recognition



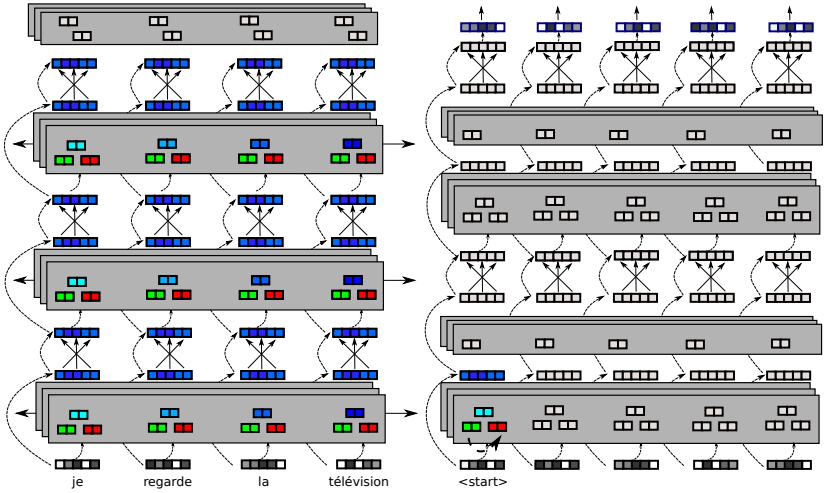
Encoder-decoder

- ‘Original’ transformer architecture, developed in context of machine translation
- Encoder builds bi-directional representations of the source; decoder generates the target left to right, with **cross-attention** to the encoder
- Can equally be used for other sequence-to-sequence tasks
- NLP tasks can be reframed as sequence-to-sequence tasks

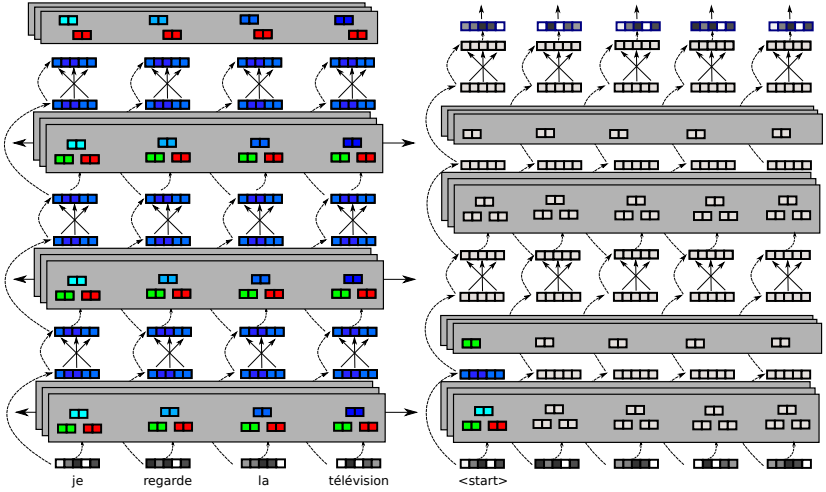
Encoder-decoder



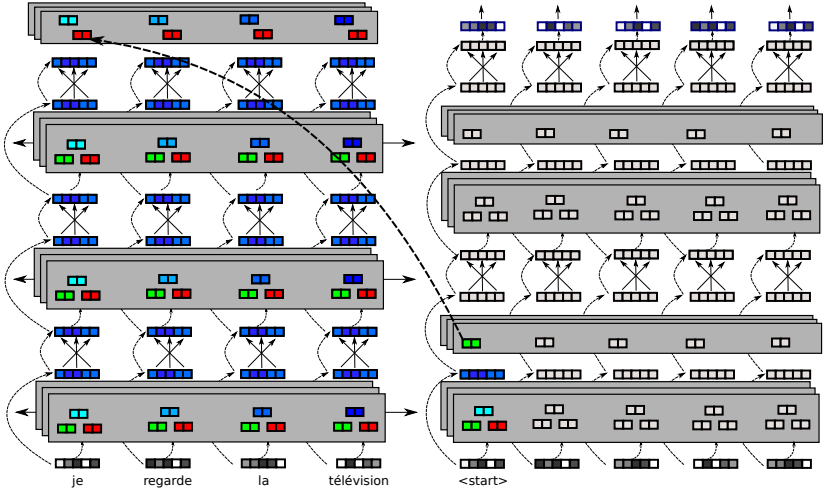
Encoder-decoder



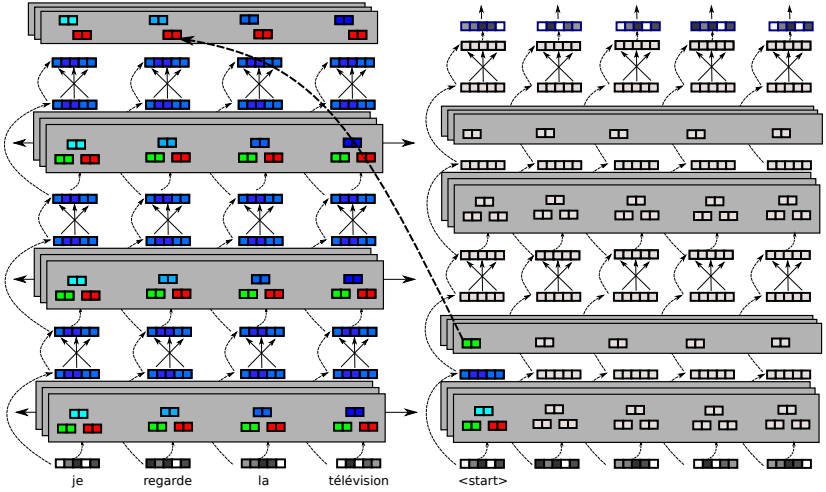
Encoder-decoder



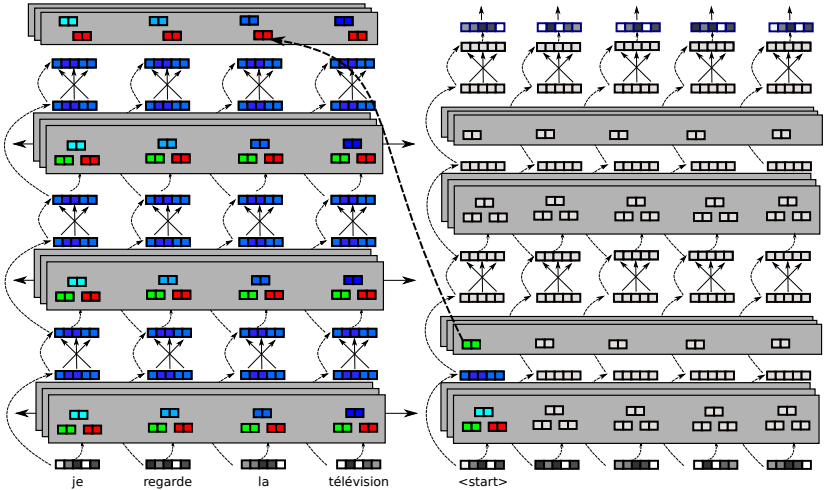
Encoder-decoder



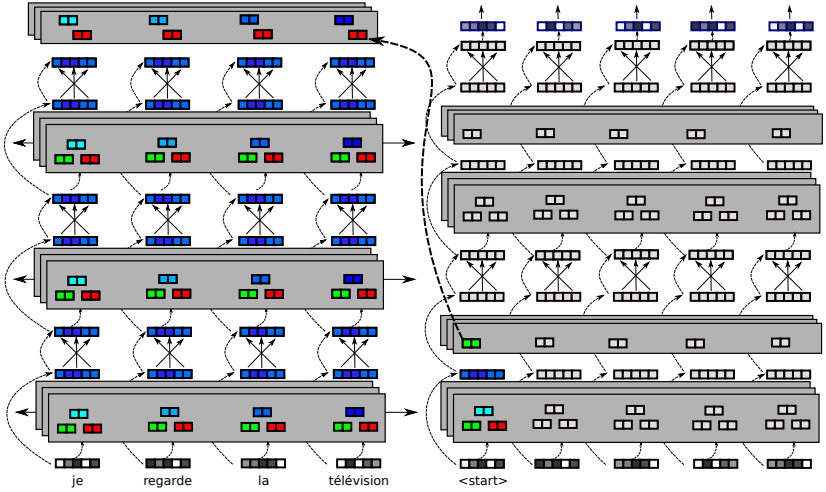
Encoder-decoder



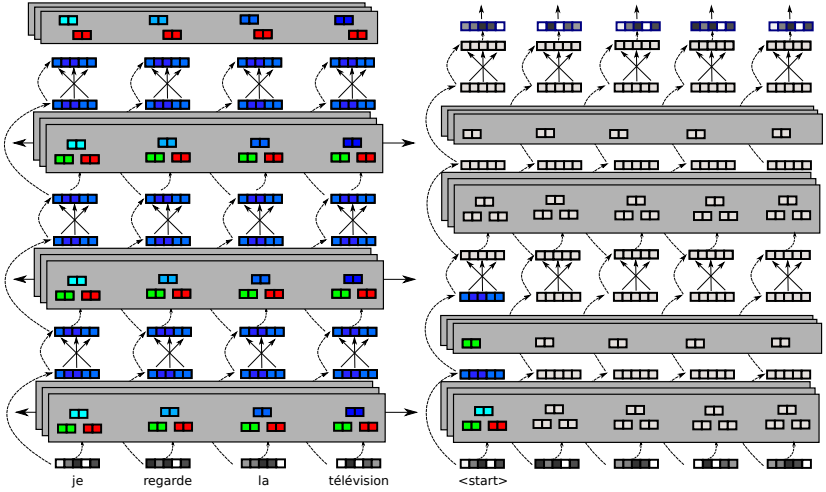
Encoder-decoder



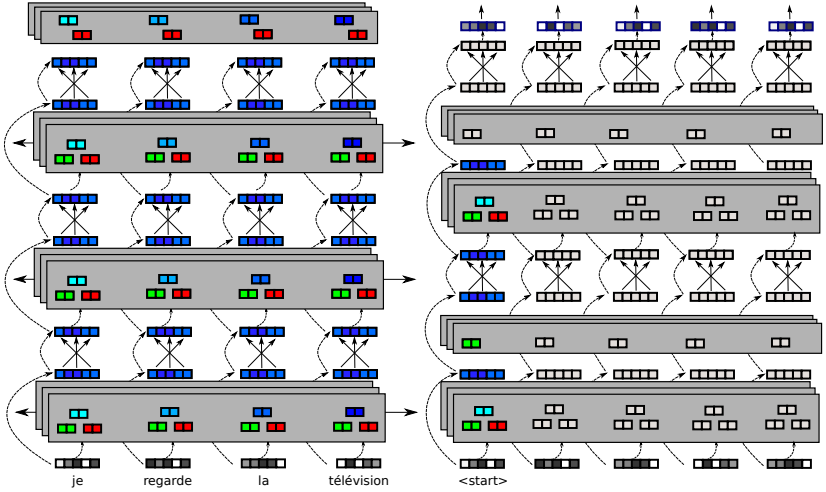
Encoder-decoder



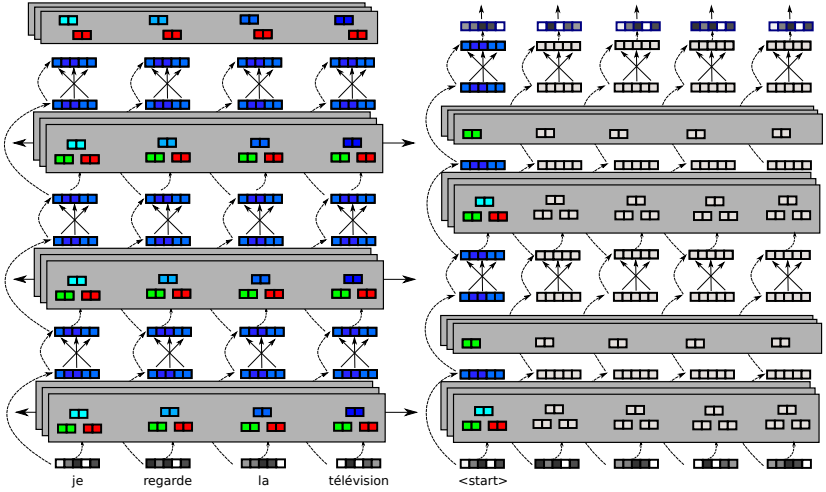
Encoder-decoder



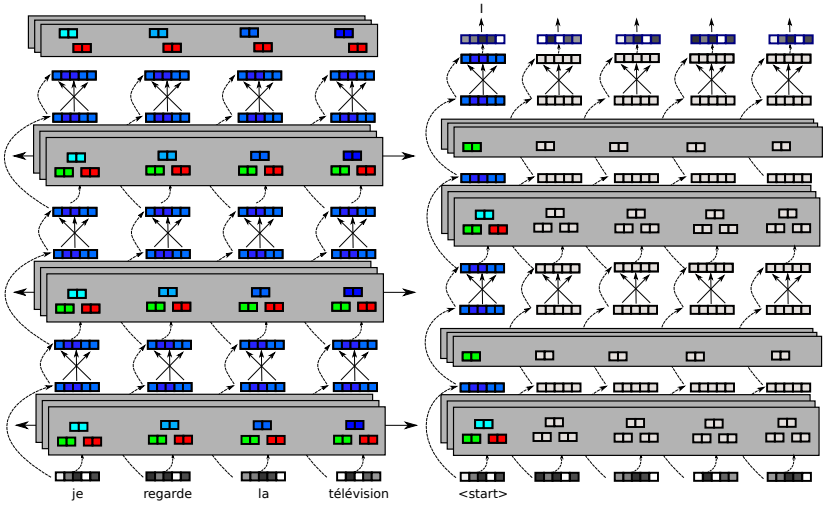
Encoder-decoder



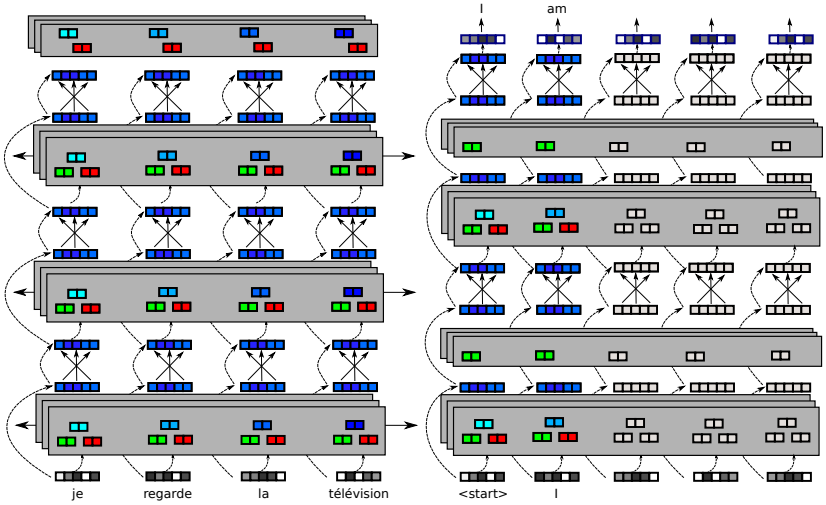
Encoder-decoder



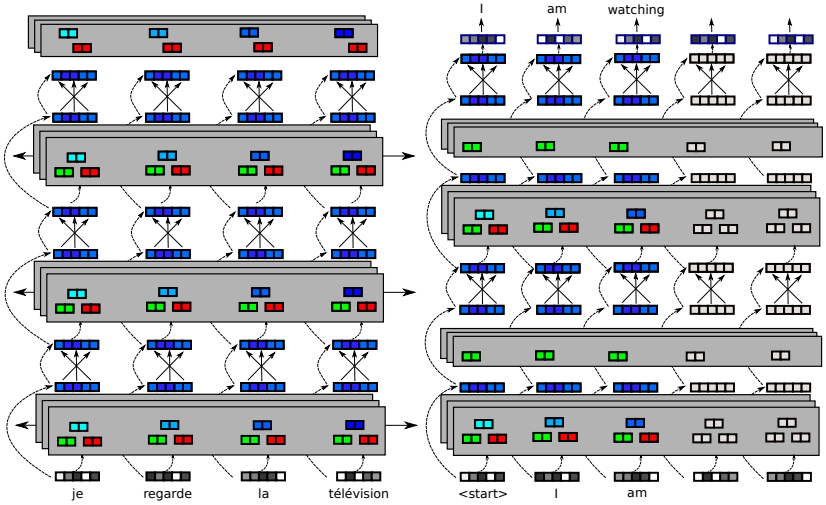
Encoder-decoder



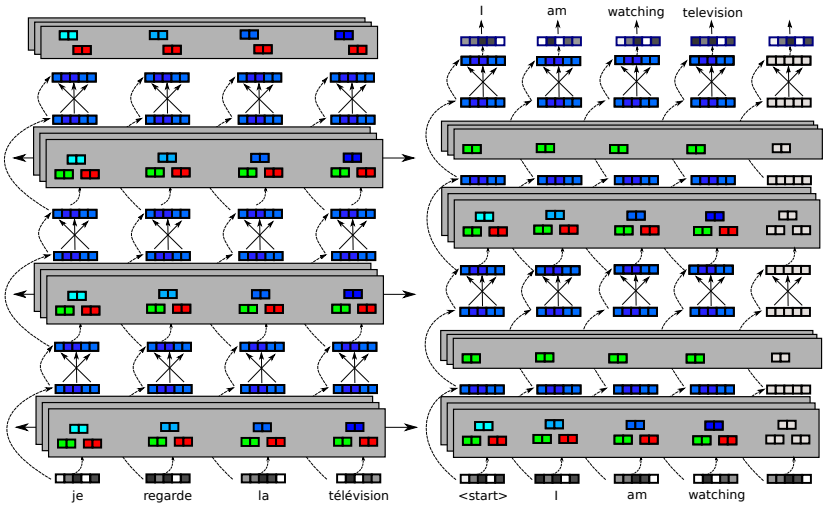
Encoder-decoder



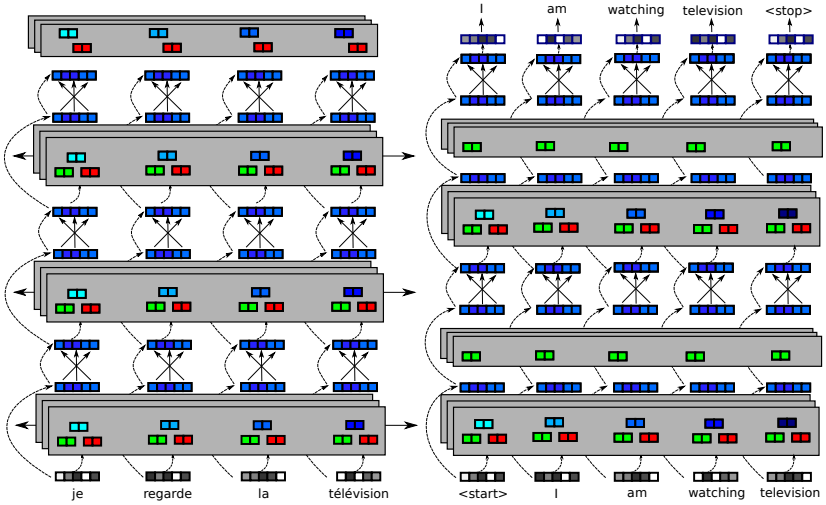
Encoder-decoder



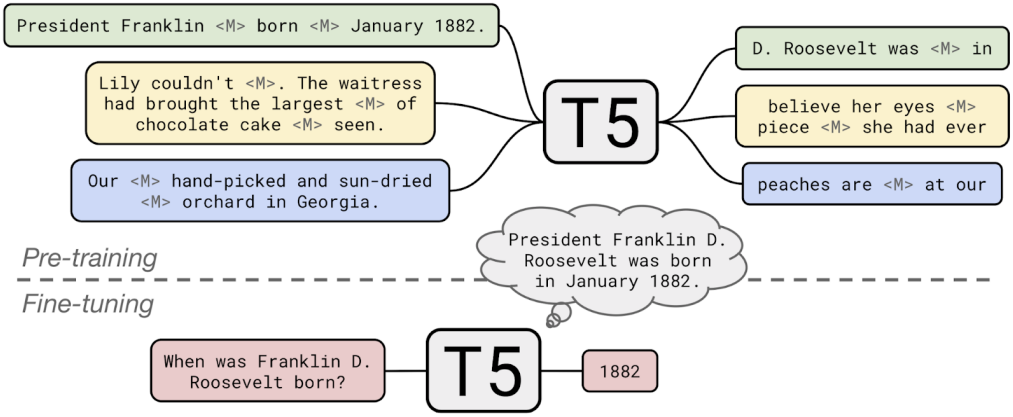
Encoder-decoder



Encoder-decoder

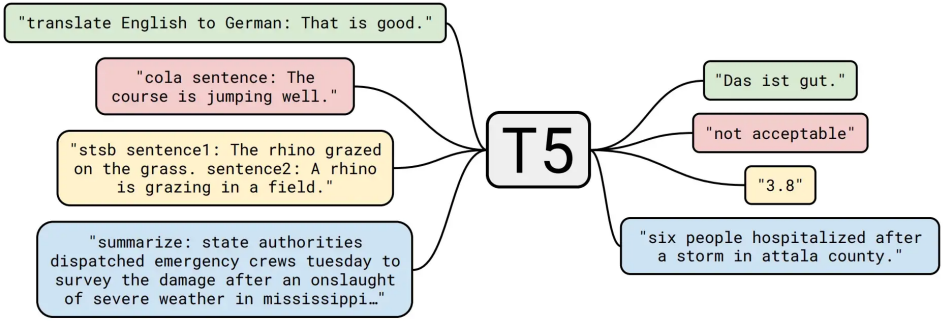


T5



Raffel et al., 2020. Exploring the limits of transfer learning with a unified text-to-text transformer.

T5



Raffel et al., 2020. Exploring the limits of transfer learning with a unified text-to-text transformer.